TECHNISCHE UNIVERSITÄT BERLIN

PROJECT REPORT

Improving Tool Invocation Reliability for LLM Agents in Remote Sensing Tasks

James Reynaldi

Matr. No. 512639

March 3, 2026

1 Introduction

Remote sensing is the science of acquiring information about the Earth's surface and atmosphere from a distance, typically using satellite or airborne sensors [6]. Some of the important applications of remote-sensing are climate change monitoring, urban growth management, disaster detection, etc [8]. Because of the advancements of remote sensing technology, dozens of terabytes of high-resolution images can be acquired everyday [9]. However, in practice, remote sensing analysis still heavily relies on manual interpretation, which hinders real time scalable analysis in operational settings [10]. These bottlenecks underscore the need for LLM-powered automation to overcome the constraints of manual interpretation

Large language models (LLMs) have demonstrated remarkable capabilities in understanding and generating natural language [5], leading to the birth of LLM-Agents, systems that leverage LLMs as their core reasoning engine to decompose complex tasks into step-by-step plans, invoke external tools, such as vision modules, calculators, and code interpreters, and reason across intermediate states to achieve user-defined goals [7]. However, recent benchmarks reveal critical weaknesses: agents struggle with tool selection accuracy and argument specification, often producing malformed or semantically incorrect tool calls [7]. These errors propagate through multi-step reasoning chains, degrading final answer quality. Existing systems lack robust mechanisms to validate and filter invalid tool predictions before execution. This paper investigates how validation mechanisms affect agent performance on the ThinkGeo benchmark. We evaluate two validation approaches: (1) *constraint-only validation*, which enforces argument schema compliance, and (2) *constraint validation combined with a learned classifier*, which additionally filters semantically implausible tool choices.

2 State of the Art

Remote Sensing ChatGPT [2] represents an early effort to apply tool-augmented LLMs to remote sensing image analysis. The system uses ChatGPT as a central controller to orchestrate seven specialized remote sensing tools, with BLIP-generated image captions [3] providing visual cues to compensate for ChatGPT's text-only nature. However, the system struggles with out-of-distribution queries (e.g., requesting segmentation of classes outside the model's training data) and occasionally hallucinates answers rather than requesting clarification. Notably, the system lacks explicit validation mechanisms to detect and reject invalid tool calls, and was not evaluated on a standardized benchmark.

RS-Agent [9] advances this paradigm with a more sophisticated architecture implementing Retrieval-Augmented Generation (RAG) for retrieving domain knowledge as well as tool selection examples, which are then added to the prompt to help the LLM make the right tool call. This approach achieves strong performance across scene classification,

visual question answering, and object counting tasks, outperforming standard LLMs and VLMs such as GeoChat and LHRS-Bot. However, RS-Agent relies primarily on prompt engineering to improve tool selection, without systematic validation of generated tool calls.

More recently, ThinkGeo [7] introduced the first benchmark for evaluating tool-augmented agents in remote sensing, comprising 486 tasks across seven application domains. Their evaluation reveals that even state-of-the-art models like GPT-4o achieve only 67.9% final answer correctness. They also found that correctly selecting which tool to use is the most important factor in getting the right answer, yet models consistently struggled with this, frequently making errors in both tool choice and argument formatting. These findings underscore that improving tool selection and reducing tool call errors remain critical challenges.

3 Methodology

In this work, the ThinkGeo framework [7] is used to evaluate the LLM Agent's performance. ThinkGeo uses the ReAct pattern (Reasoning + Acting), in which the agent alternates between generating intermediate thoughts and invoking external tools, as illustrated in Figure 1.:

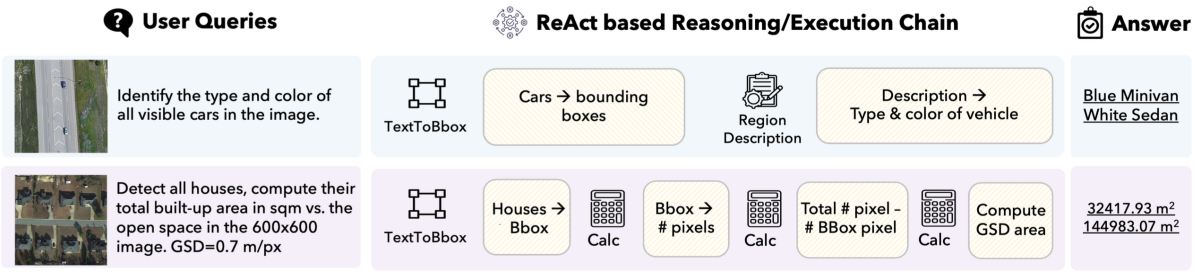

Figure 1: ReAct Pattern [7]

At the initial step, the agent receives the user query, prompt, and image as input. For subsequent steps, the agent processes the accumulated history—which includes the original user query, previous thoughts, tool choices, arguments, and image context—along with the prompt.The LLM then produces either a tool call (with arguments) or a final answer. Each new thought, tool choice, and argument is appended to the history, enabling multi-step reasoning and execution.

We evaluate two QWEN models: *Qwen 1.5-7B-Chat* [1] and *Qwen 2.5-7B-Instruct* [11]. Their baseline performance is shown in Table 1.

Here, *Inst_align* measures the alignment of the response types of the agent, *Tool_acc*, *Arg_acc*, *Ans_acc* measures correctness of tool choice, argument and answer respectively, and *Tool_call_error_rate* quantifies the proportion of steps resulting in tool call errors. High error rates are typical for open-source LLMs without validation, due to

Model	Inst_align	Tool_acc	Arg_acc	Ans_acc	Tool_call_error_rate
QWEN1.5-7b-chat	26.81%	10.21%	3.68%	47.70%	77%
QWEN2.5-7b-instruct	67.63%	53.63%	21.94%	61.67%	22%

Table 1: Baseline Performance of QWEN models

both format and semantic errors. Tool call errors are categorized as *ARGS_ERROR* (invalid argument formats or schema mismatches) and *API_ERROR* (empty tool calls or non-existent tools).

3.1 Proposed Method

To improve the tool choice accuracy and reduce the tool call errors, the following workflow shown in Figure 2 was proposed:

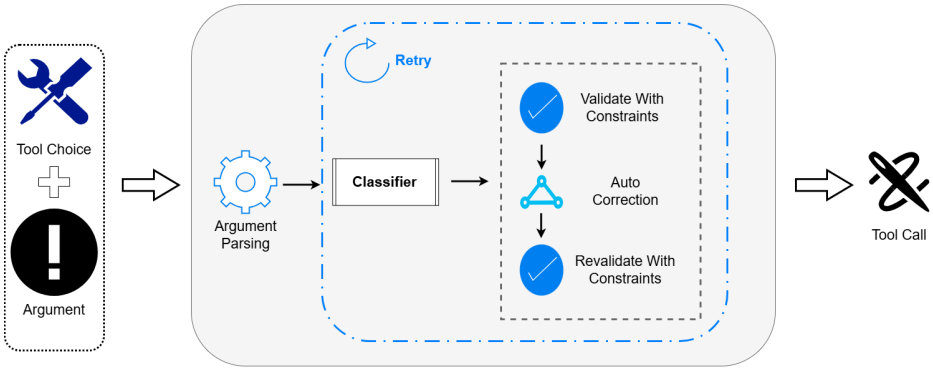

Figure 2: Validation Workflow

At each step, the tool choice and arguments are passed to a validator. The validation process consists of:

1. **Argument Parsing:** Parses and corrects invalid syntax (e.g., converting markdown or fixing bracket types).
2. **Classifier Validation:** A binary classifier checks whether the tool choice is semantically plausible given the context, and rejects if it does not.
3. **Constraint Validation:** If the tool passes the classifier, argument constraints are checked (format, type, required fields). Constraints do not check semantic correctness.
4. **Auto-Correction and Revalidation:** If constraints are not met, an auto-correction is attempted, followed by revalidation. If revalidation fails, the arguments are rejected.

5. **Retry Logic:** If validation fails (either tool choice rejection by classifier or argument rejection due constraint violation), the agent is prompted to retry (up to a fixed limit; in the experiment, 5 retries were used). If all retries are exhausted, a tool call error is recorded for that step.

Figure 3 shows the classifier architecture:

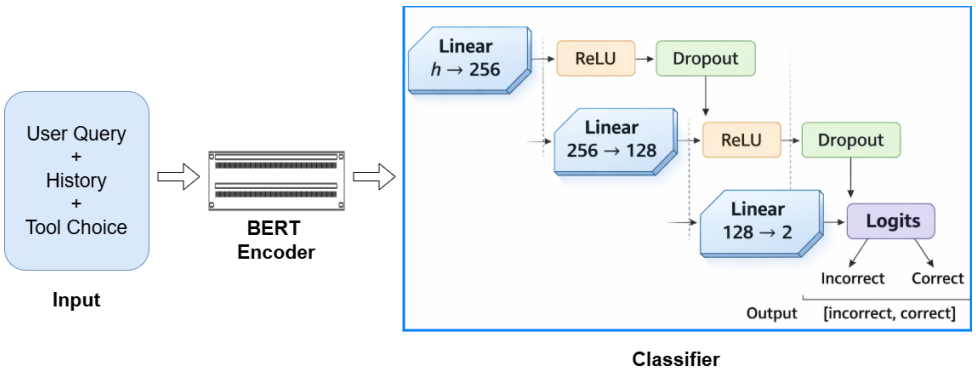

Figure 3: Classifier Architecture

The classifier receives the concatenated user query, tool call history, and selected tool choice, which are encoded by a pretrained BERT model. The resulting embedding is passed through a three-layer feedforward neural network (hidden sizes 256 and 128), producing two logits. After softmax, these yield probabilities for the tool call being correct or incorrect. Tool choices are accepted if $P(\text{correct})$ exceeds a confidence threshold τ.

4 Results

Table 2 presents the performance of QWEN models under two validation regimes: constraint-only validation and constraint validation combined with the classifier.

Table 2: Performance comparison of QWEN models under different conditions

Model	**Condition**	**Inst. align**	**Tool_acc**	**Arg_acc**	**Ans_acc**	**Tool_call_error rate**
QWEN1.5-7b-chat	Constraint Only	31.00% +4.19%	7.56% - 2.65%	3.32% - 0.36%	49.04% +1.34%	0.00%
	With Classifier	63.53% +36.72%	28.90% +18.69%	9.09% +5.41%	50.29% +2.59%	22.38%
QWEN2.5-7b-Instruct	Constraint Only	95.95% +28.32%	73.14% +19.51%	28.68% +6.74%	62.21% +0.54%	0.00%
	With Classifier	98.59% +30.96%	78.43% +24.80%	31.98% +10.04%	64.30% +2.63%	20.91%

4.1 Constraint-Only Validation

For both QWEN1.5 and QWEN2.5, constraint validation combined with argument parsing eliminates all tool call errors, resulting in a 0.00% tool_call_error rate. This indicates that both models can produce properly formatted tool calls that satisfy schema requirements. However, this does not guarantee semantic correctness, as constraint validation only checks format, type compliance, and required fields, not whether the chosen tool and argument are appropriate for the task.

For QWEN2.5, constraint validation significantly improves inst_align (+28.32%), tool_acc (+19.51%), and arg_acc (+6.74%), demonstrating the model's capability to produce correct tool calls when guided by schema constraints. In contrast, QWEN1.5 shows only modest improvement in inst_align (+4.19%), while tool_acc actually decreases slightly (-2.65%). This occurs because, although more steps are now valid (correct response type, proper format), the model frequently selects the wrong tool. The 0% error rate for QWEN1.5 reflects format compliance, not task correctness—the model achieves only 7.56% tool accuracy despite producing no validation errors.

For both models, answer accuracy (ans_acc) shows minimal improvement, as the proposed validation approach primarily addresses tool call quality and does not directly improve final answer generation.

4.2 Constraint Validation with Classifier

Adding the learned classifier on top of constraint validation introduces tool call errors (22.38% for QWEN1.5, 20.91% for QWEN2.5), as the classifier rejects semantically implausible tool choices. Analysis reveals that the classifier achieves perfect precision in rejection: all tool calls it rejects are indeed incorrect, and it never rejects correct tool calls.

All metrics improve substantially with classifier validation. For QWEN1.5, tool_acc increases from 7.56% to 28.90%, and inst_align jumps from 31.00% to 63.53%. For QWEN2.5, tool_acc improves from 73.14% to 78.43%, and inst_align reaches 98.59%.

This improvement is primarily due to a *selective evaluation effect*: the classifier filters out poor predictions and hard cases where the model would likely fail, leaving only higher-quality predictions for evaluation. This raises reported metrics but does not reflect genuine model capability improvement, it demonstrates rather that the classifier successfully identifies and rejects low-confidence or implausible tool choices before they are measured.

Despite these improvements, both models still exhibit substantial room for improvement in tool accuracy (28.90% for QWEN1.5, 78.43% for QWEN2.5), indicating that many incorrect tool calls still pass validation. Table 5 shows the most common tool selection

errors that were not caught by the classifier.

5 Future Work

While constraint validation in this work ensures that tool choices and argument formats are syntactically correct, it does not address the semantic correctness of the arguments—i.e., whether the arguments are appropriate for the user query and task context.

In order to generate semantically correct arguments, we can finetune an LLM on a language modeling task, or use a self-verification method that generates contrastive questions to choose between candidate parameter values, ensuring alignment with the user query through explicit reasoning [4].

In cases of rejected tool choice, we can also train a tool classifier to suggest the correct tool instead of leaving them as errors, or use ToolVerifier's [4] contrastive verification approach that generates targeted questions to distinguish between the top two candidate tools, then answers these questions to resolve ambiguity and select the most appropriate tool.

Additionally, more advanced classifier architectures—such as those incorporating cross-attention—could be investigated to reduce the rate of wrongly accepted tool calls and further improve tool selection accuracy.

References

[1] Jinze Bai et al. *Qwen Technical Report*. 2023. arXiv: 2309.16609 [cs.CL]. URL: https://arxiv.org/abs/2309.16609.

[2] Haonan Guo et al. *Remote Sensing ChatGPT: Solving Remote Sensing Tasks with ChatGPT and Visual Models*. 2024. arXiv: 2401.09083 [cs.CV]. URL: https://arxiv.org/abs/2401.09083.

[3] Junnan Li et al. *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. 2022. arXiv: 2201.12086 [cs.CV]. URL: https://arxiv.org/abs/2201.12086.

[4] Dheeraj Mekala et al. "TOOLVERIFIER: Generalization to New Tools via Self-Verification". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 5026–5041. DOI: 10.18653/v1/2024.findings-emnlp.289. URL: https://aclanthology.org/2024.findings-emnlp.289/.

[5] Humza Naveed et al. *A Comprehensive Overview of Large Language Models*. 2024. arXiv: 2307.06435 [cs.CL]. URL: https://arxiv.org/abs/2307.06435.

[6] Robert A. Schowengerdt. *Remote Sensing - Models and Methods for Image Processing (3rd Edition)*. eng. Third edition. Amsterdam ; Tokyo: Elsevier, 2007. ISBN: 9780123694072.

[7] Akashah Shabbir et al. *ThinkGeo: Evaluating Tool-Augmented Agents for Remote Sensing Tasks*. 2025. arXiv: 2505.23752 [cs.CV]. URL: https://arxiv.org/abs/2505.23752.

[8] Wenjia Xu et al. *RS-Agent: Automating Remote Sensing Tasks through Intelligent Agent*. 2025. arXiv: 2406.07089 [cs.CV]. URL: https://arxiv.org/abs/2406.07089.

[9] Wenjia Xu et al. *RS-Agent: Automating Remote Sensing Tasks through Intelligent Agent*. 2026. arXiv: 2406.07089 [cs.CV]. URL: https://arxiv.org/abs/2406.07089.

[10] Xing Yan et al. "Real-Time Remote Sensing for Sudden Surface Anomalies: A Review of Principles and Challenges". In: *Space: Science & Technology* 5 (2025), p. 0362. DOI: 10.34133/space.0362. eprint: https://spj.science.org/doi/pdf/10.34133/space.0362. URL: https://spj.science.org/doi/abs/10.34133/space.0362.

[11] An Yang et al. *Qwen2 Technical Report*. 2024. arXiv: 2407.10671 [cs.CL]. URL: https://arxiv.org/abs/2407.10671.

Tool*	Required Fields	Optional Fields	Constraints
Calc	Expression	-	Must contain valid math expression (regex: `[0-9+*/( )%]+`)
TTB	Image, text, top1	-	image/text must be strings; top1 must be boolean
Draw	Image, bbox	-	image is string; bbox must be valid (4 coordinates)
CGO	Image, text	Bbox	image/text are strings; bbox (if present) must be valid format (4 coordinates)
RAD	Image, bbox, attribute	-	All required; bbox must be valid
Plot	Command	-	command must be string
Solv	Command	-	command must be string
GS	query	k	query must be string
NA			NoActions not allowed - Retry

Table 3: *

*Tool abbreviations: Calc=Calculator, TTB=TextToBbox, Draw=DrawBox, CGO=CountGivenObject, RAD=RegionAttributeDescription, Solv=Solver, GS=GoogleSearch, NA=NoAction

Table 4: Tools specification and constraints

Table 5: Tool Mismatches

Expected Tool	Predicted Tool	Count
Calculator	DrawBox	21
Calculator	RegionAttributeDescription	17
RegionAttributeDescription	TextToBbox	17
TextToBbox	Calculator	14
Calculator	TextToBbox	14
Calculator	CountGivenObject	11
TextToBbox	RegionAttributeDescription	9
RegionAttributeDescription	CountGivenObject	9
TextToBbox	CountGivenObject	9